ECE 3849 Real-Time Embedded Systems

Lab 4: Spectrum Analyzer and DMA

Rachel Turman, Tracy Yang

Signoff TA and Date: Eric 5/5/25

Introduction:

In this lab, the objective was to add a spectrum analyzer mode to the oscilloscope and to optimize the ADC I/O using DMA (Direct Memory Access). The real-time system we have been building upon needed to switch between oscilloscope mode and spectrum mode through the push of a button. We improved on the oscilloscope through transferring, accessing, and sharing memory without any data corruption. From this, we were able to successfully implement a new mode as well as optimize processes.

Discussion & Results:

We decided to first implement the spectrum mode using the Kiss FFT library to compute the frequency spectrum of the ADC samples. Initially, to improve the frequency resolution and reduce spectral leakage, we implemented a Blackman window. However, after testing, we found that the spectrum display was acceptable without the windowing. For this reason, the Blackman windowing was not included in the final implementation, simplifying the computation and avoiding additional CPU overhead.

By adding on to our button_init(), processingTask(), and displayingTask() functions, we could switch between oscilloscope and spectrum mode. With the implementation, we could continue using waveform display in the time-domain while showing a frequency-domain representation.

While we had troubles with importing the library, halting our progress for a while due to troubleshooting and debugging, we eventually were able to complete spectrum mode and move on to ADC optimization. We incorporated DMA to handle ADC data transfer. Because of this, it greatly reduced the need for the CPU to constantly poll or interrupt to retrieve samples. The difference in CPU Load can be seen in the following table:

Table 1. CPU Load Measurements of the System

ADC Configuration	Sampling Rate	CPU Load	ISR Relative Deadline
Single Sample ISR	1msps	75.88	1 microsecond
DMA	1msps	1.80	1,024 microseconds
DMA	2msps	2.40	512 microseconds

The DMA was configured in ping-pong mode using two buffer halves, allowing continuous data to be acquired so as to not miss samples. Thos means that it goes between a primary and alternate channel. If none of the channels are available, it restarts the DMA entirely. Performance significantly improved when DMA was implemented, going from 75.88% CPU Load to 1.8%. While this enhancement initially seemed incorrect and suspiciously large, further testing and confirmation from the TA affirmed the value. The change allowed for the remaining CPU workload to be allocated to display rendering and update the user interface.

Conclusion:

At the end of this lab, we were able to successfully extend the oscilloscope system with a spectrum analyzer mode and significantly optimized ADC performance using DMA. Offloading repetitive I/O tasks to the DMA controller dramatically reduced the CPU Load, improving the system's efficiency and freeing resources for other tasks. The implementation highlighted the importance of properly applying signal processing techniques when working in the frequency domain. Overall, the system now provides flexible, real-time analysis capabilities in both time and frequency domains.